

```

#include <iostream>
#include <string>
#include <vector>
#include <algorithm> // For reverse
using namespace std;

class Digit {
    int digit;
public:
    Digit(int d = 0) : digit(d % 10) {} // default argument provided
    void setDigit(int d) { digit = d % 10; }
    int getDigit() const { return digit; }
};

class Integer {
    vector<Digit> value;
    bool sign;
public:
    Integer(string n);
    Integer();
    Integer addition(const Integer &b) const;
    Integer subtraction(const Integer &b) const;
    void display() const;
    bool operator>=(const Integer &other) const;
    bool operator<(const Integer &other) const;
    bool operator<=(const Integer &other) const;
    bool operator>(const Integer &other) const;
    bool operator==(const Integer &other) const;
    bool operator!=(const Integer &other) const;
};

Integer::Integer(string n) {
    int len = n.length();
    int start = 0;
    sign = true;

    if (n[0] == '-') {
        sign = false;
        start = 1;
    }

    for (int i = len - 1; i >= start; --i) { // Start from the end to maintain correct order
        value.push_back(Digit(n[i] - '0'));
    }
}

Integer::Integer() {
    sign = true;
}

Integer Integer::addition(const Integer &b) const {
    Integer result;
    int carry = 0;
    int maxSize = max(value.size(), b.value.size());

    result.value.resize(maxSize);

    for (size_t i = 0; i < maxSize; ++i) {
        int sum = carry;
        if (i < value.size()) sum += value[i].getDigit();
    }
}

```

```

        if (i < b.value.size()) sum += b.value[i].getDigit();
        result.value[i].setDigit(sum % 10);
        carry = sum / 10;
    }

    // If there is a remaining carry, add it to the result
    if (carry != 0) {
        result.value.push_back(Digit(carry));
    }

    return result;
}

Integer Integer::subtraction(const Integer &b) const {
    Integer result;
    Integer larger, smaller;
    int borrow = 0;

    if (*this >= b) {
        larger = *this;
        smaller = b;
    } else {
        result.sign = false;
        larger = b;
        smaller = *this;
    }

    int maxSize = larger.value.size();

    result.value.resize(maxSize);

    for (size_t i = 0; i < maxSize; ++i) {
        int diff = borrow;
        if (i < larger.value.size()) diff += larger.value[i].getDigit();
        if (i < smaller.value.size()) diff -= smaller.value[i].getDigit();
        if (diff < 0) {
            diff += 10;
            borrow = 1;
        } else {
            borrow = 0;
        }
        result.value[i].setDigit(diff);
    }

    // Remove leading zeros
    while (!result.value.empty() && result.value.back().getDigit() == 0) {
        result.value.pop_back();
    }

    return result;
}

bool Integer::operator>=(const Integer &other) const {
    if (sign && !other.sign) return true;
    if (!sign && other.sign) return false;
    if (value.size() > other.value.size()) return true;
    if (value.size() < other.value.size()) return false;
    for (int i = value.size() - 1; i >= 0; --i) {
        if (value[i].getDigit() > other.value[i].getDigit()) return true;
        if (value[i].getDigit() < other.value[i].getDigit()) return false;
    }
    return true;
}

```

```

    }
    return true;
}

bool Integer::operator<(const Integer &other) const {
    return !(*this >= other);
}

bool Integer::operator<=(const Integer &other) const {
    return (*this < other) || (*this == other);
}

bool Integer::operator>(const Integer &other) const {
    return !(*this <= other);
}

bool Integer::operator==(const Integer &other) const {
    if (sign != other.sign) return false;
    if (value.size() != other.value.size()) return false;
    for (size_t i = 0; i < value.size(); ++i) {
        if (value[i].getDigit() != other.value[i].getDigit()) return false;
    }
    return true;
}

bool Integer::operator!=(const Integer &other) const {
    return !(*this == other);
}

void Integer::display() const {
    if (!sign)
        cout << "-";

    if (value.empty()) {
        cout << 0;
        return;
    }

    for (auto it = value.rbegin(); it != value.rend(); ++it) { // Reverse iteration for correct order
        cout << it->getDigit();
    }
}

```